QuickSight Dashboard Queries — Bear EHR

> *Generated: 2026-03-31 Database: MariaDB 10.2.29 Source: Bear EHR metric reports (* `app/models/metric/` *)*
> *All queries use parameterized date range:* `@from_date` *and* `@until_date`

Table of Contents

1. Top 10 CPT Codes (by Superbill & by Collection)
2. No-Show Rates
3. Top 10 Payors (with Payments, Adjustments, Denials)
4. Location Performance — Revenue per Location
5. Provider Performance — Revenue per Provider
6. Clients per Zip Code
7. Diagnosis per Zip Code
8. Suggested Analyses (5 Graphs)

Parameter Setup

In QuickSight, create two date parameters:

```
-- Replace these with QuickSight parameters
SET @from_date  = '2025-01-01';
SET @until_date = '2025-12-31';
```

---

1. Top 10 CPT Codes

1a. Top 10 CPT Codes by Superbill Volume

```
-- Top 10 CPT codes by number of procedures on superbills
--
-- Join chain:
--   procedures → fees          (fee schedule: rate, description, CPT code)
--   procedures → superbills    (the clinical billing document linking encounter to
charges)
--   superbills are filtered by reported_start (date of service on the superbill)
--
-- Note: procedure_string on procedures is the denormalized CPT code + modifiers
SELECT
    pr.procedure_string                                          AS cpt_code,
    fe.description                                               AS
service_description,
    COUNT(DISTINCT pr.id)                                        AS procedure_count,
    SUM(pr.units)                                                AS total_units,
    -- charges = fee rate × units (what the fee schedule says we should charge)
    ROUND(SUM(CAST(fe.rate_cents AS SIGNED) / 100.0 * pr.units), 2) AS
fee_schedule_charges
FROM procedures AS pr
INNER JOIN fees       AS fe ON pr.fee_id = fe.id
```

```
INNER JOIN superbills AS sb ON pr.superbill_id = sb.id
-- Filter by superbill date of service
WHERE sb.reported_start BETWEEN CAST(@from_date AS DATETIME)
                              AND CAST(CONCAT(@until_date, ' 23:59:59') AS DATETIME)
GROUP BY pr.procedure_string, fe.description
ORDER BY procedure_count DESC
LIMIT 10;
```

1b. Top 10 CPT Codes by Collection (Payments Received)

```
-- Top 10 CPT codes ranked by actual money collected (paid + copaid + client_paid)
--
-- Join chain:
--   procedures → fees                    (fee schedule data)
--   procedures → superbills              (clinical billing document)
--   procedures → charges_procedures      (HABTM join: procedures link to charge
line items)
--   charges_procedures → charges         (actual billing line items with payment
tracking)
--
-- Financial columns on charges (all stored as _cents integers, divide by 100):
--   amount_cents      = billed amount (what was submitted to payor)
--   paid_cents        = insurance payment received
--   copaid_cents      = copay collected
--   client_paid_cents = other client payments
--   adjusted_cents    = contractual adjustments
--   denied_cents      = denied amounts
--   balance_cents     = outstanding balance
SELECT
    pr.procedure_string                                          AS cpt_code,
    fe.description                                               AS
service_description,
    COUNT(DISTINCT pr.id)                                        AS procedure_count,
    ROUND(SUM(CAST(ch.amount_cents AS SIGNED) / 100.0), 2)     AS total_billed,
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0), 2)       AS insurance_paid,
    -- copaid + client_paid = total client-side collections
    ROUND(SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0), 2) AS
client_collections,
    -- revenue = insurance paid + all client collections
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0), 2) AS total_revenue,
    -- collection rate = revenue / billed
    ROUND(
        (SUM(CAST(ch.paid_cents AS SIGNED)) + SUM(CAST(ch.copaid_cents AS SIGNED))
         + SUM(CAST(ch.client_paid_cents AS SIGNED)))
        * 100.0 / NULLIF(SUM(CAST(ch.amount_cents AS SIGNED)), 0), 1
    )                                                            AS
collection_rate_pct
FROM procedures AS pr
```

```
INNER JOIN fees                AS fe   ON pr.fee_id = fe.id
INNER JOIN superbills          AS sb   ON pr.superbill_id = sb.id
-- charges_procedures is a HABTM join table (no model) linking procedures ↔ charges
LEFT  JOIN charges_procedures  AS chpr ON pr.id = chpr.procedure_id
LEFT  JOIN charges             AS ch   ON chpr.charge_id = ch.id
WHERE sb.reported_start BETWEEN CAST(@from_date AS DATETIME)
                            AND CAST(CONCAT(@until_date, ' 23:59:59') AS DATETIME)
GROUP BY pr.procedure_string, fe.description
ORDER BY total_revenue DESC
LIMIT 10;
```

---

2. No-Show Rates

```
-- Appointment show/no-show/cancellation rates
--
-- Bear appointments do NOT have a status column.
-- Status is derived from timestamp columns:
--   checked_in_at  IS NOT NULL → client showed up (checked in)
--   checked_out_at IS NOT NULL → visit completed
--   noshow_at      IS NOT NULL → marked as no-show
--   cancelled_at   IS NOT NULL → cancelled (cancel_late flag distinguishes late
cancels)
--   deleted_at     IS NOT NULL → soft-deleted (excluded)
--
-- event_type_id filters to actual appointments (excludes blocks, notes, etc.)
-- EventType::APPOINTMENT_TYPE_ID = 1 in production
SELECT
    COUNT(app.id)                                              AS
total_appointments,
    -- Attended: checked in (regardless of checkout status)
    SUM(IF(app.checked_in_at IS NOT NULL, 1, 0))              AS attended,
    -- No-shows
    SUM(IF(app.noshow_at IS NOT NULL, 1, 0))                  AS no_shows,
    -- Late cancellations (cancelled + cancel_late flag = true)
    SUM(IF(app.cancelled_at IS NOT NULL
           AND app.cancel_late = 1, 1, 0))                    AS late_cancellations,
    -- Regular cancellations
    SUM(IF(app.cancelled_at IS NOT NULL
           AND app.cancel_late = 0, 1, 0))                    AS cancellations,
    -- Rates as percentages
    ROUND(SUM(IF(app.noshow_at IS NOT NULL, 1, 0))
          * 100.0 / COUNT(app.id), 1)                         AS noshow_rate_pct,
    ROUND(SUM(IF(app.cancelled_at IS NOT NULL, 1, 0))
          * 100.0 / COUNT(app.id), 1)                         AS cancel_rate_pct,
    ROUND(SUM(IF(app.checked_in_at IS NOT NULL, 1, 0))
          * 100.0 / COUNT(app.id), 1)                         AS show_rate_pct
FROM appointments AS app
WHERE app.event_type_id = 1
  AND app.deleted_at IS NULL
  AND app.date_time_starts BETWEEN CAST(@from_date AS DATETIME)
```

```
                                AND CAST(CONCAT(@until_date, ' 23:59:59') AS
DATETIME);
```

2b. No-Show Rates by Month (for trend line)

```
-- Monthly no-show trend — use for a line chart in QuickSight
SELECT
    DATE_FORMAT(app.date_time_starts, '%Y-%m')                AS month,
    COUNT(app.id)                                             AS total,
    SUM(IF(app.checked_in_at IS NOT NULL, 1, 0))             AS attended,
    SUM(IF(app.noshow_at IS NOT NULL, 1, 0))                 AS no_shows,
    SUM(IF(app.cancelled_at IS NOT NULL, 1, 0))              AS cancelled,
    ROUND(SUM(IF(app.noshow_at IS NOT NULL, 1, 0))
          * 100.0 / COUNT(app.id), 1)                        AS noshow_rate_pct
FROM appointments AS app
WHERE app.event_type_id = 1
  AND app.deleted_at IS NULL
  AND app.date_time_starts BETWEEN CAST(@from_date AS DATETIME)
                                AND CAST(CONCAT(@until_date, ' 23:59:59') AS
DATETIME)
GROUP BY month
ORDER BY month;
```

---

3. Top 10 Payors

```
-- Top 10 insurance payors with payments, adjustments, and denials
--
-- Join chain (follows Bear metric pattern):
--   procedures → superbills              (clinical document with date_of_service)
--   superbills → insurance_policies      (first_insurance_id = primary insurance on
superbill)
--   insurance_policies → companies       (payor_id → companies table, type =
'Payor')
--   procedures → charges_procedures → charges  (billing line items with financial
breakdown)
--   superbills → encounter_forms → encounters → appointments  (for encounter/visit
counts)
--
-- Adjustment and denial amounts come from denormalized _cents columns on charges.
-- These are populated when EOBs (Explanation of Benefits) are processed.
SELECT
    pa.name                                                   AS payor,
    COUNT(DISTINCT ef.id)                                     AS encounters,
    COUNT(DISTINCT app.id)                                    AS appointments,
    -- Fee schedule charges (rate × units)
    ROUND(SUM(CAST(fe.rate_cents AS SIGNED) / 100.0 * pr.units), 2) AS fee_charges,
    -- Contracted amount (negotiated rate × units)
    ROUND(SUM(CAST(fe.contracted_rate_cents AS SIGNED) / 100.0 * pr.units), 2) AS
contracted,
```

```sql
    -- What was actually billed (submitted on claim)
    ROUND(SUM(CAST(ch.amount_cents AS SIGNED) / 100.0), 2)    AS billed,
    -- Insurance payments received
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0), 2)      AS paid,
    -- Copay + client payments
    ROUND(SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0), 2) AS
client_collections,
    -- Total revenue (insurance + client)
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0), 2) AS total_revenue,
    -- Adjustments (contractual write-downs from payor)
    ROUND(SUM(CAST(ch.adjusted_cents AS SIGNED) / 100.0), 2)  AS adjustments,
    -- Denials (payor refused to pay)
    ROUND(SUM(CAST(ch.denied_cents AS SIGNED) / 100.0), 2)    AS denials,
    -- Write-offs (provider decision to forgive balance)
    ROUND(SUM(CAST(ch.writeoff_cents AS SIGNED) / 100.0), 2)  AS writeoffs,
    -- Outstanding balance
    ROUND(SUM(CAST(ch.balance_cents AS SIGNED) / 100.0), 2)   AS balance,
    -- Collection rate
    ROUND(
        (SUM(CAST(ch.paid_cents AS SIGNED)) + SUM(CAST(ch.copaid_cents AS SIGNED))
         + SUM(CAST(ch.client_paid_cents AS SIGNED)))
        * 100.0 / NULLIF(SUM(CAST(ch.amount_cents AS SIGNED)), 0), 1
    )                                                          AS
collection_rate_pct
FROM procedures AS pr
INNER JOIN fees                AS fe   ON pr.fee_id = fe.id
INNER JOIN superbills          AS sb   ON pr.superbill_id = sb.id
-- Primary insurance on the superbill
INNER JOIN insurance_policies  AS ip   ON sb.first_insurance_id = ip.id
-- Payor company (insurance carrier)
INNER JOIN companies           AS pa   ON ip.payor_id = pa.id
-- Provider (attending) via polymorphic people table
INNER JOIN people              AS pp   ON sb.attending_provider_id =
pp.identifiable_id
                                       AND pp.identifiable_type = 'User'
INNER JOIN locations           AS loc  ON sb.location_id = loc.id
-- Charges: LEFT JOIN because some procedures may not yet have charges (unbilled)
LEFT  JOIN charges_procedures  AS chpr ON pr.id = chpr.procedure_id
LEFT  JOIN charges             AS ch   ON chpr.charge_id = ch.id
-- Encounter chain for visit counts
LEFT  JOIN encounter_forms     AS ef   ON sb.encounter_form_id = ef.id
LEFT  JOIN encounters          AS en   ON ef.encounter_id = en.id
LEFT  JOIN appointments        AS app  ON en.appointment_id = app.id
WHERE sb.reported_start BETWEEN CAST(@from_date AS DATETIME)
                            AND CAST(CONCAT(@until_date, ' 23:59:59') AS DATETIME)
GROUP BY pa.name
ORDER BY total_revenue DESC
LIMIT 10;
```

4. Location Performance

```sql
-- Revenue and encounter volume by location (office/site)
--
-- Same join chain as payor performance, grouped by location instead.
-- Locations are physical office sites (e.g., Albuquerque, Santa Fe).
-- superbills.location_id identifies where the service was rendered.
SELECT
    loc.name                                                    AS location,
    COUNT(DISTINCT ef.id)                                       AS encounters,
    COUNT(DISTINCT app.id)                                      AS appointments,
    COUNT(DISTINCT sb.client_id)                                AS unique_clients,
    ROUND(SUM(CAST(fe.rate_cents AS SIGNED) / 100.0 * pr.units), 2) AS fee_charges,
    ROUND(SUM(CAST(ch.amount_cents AS SIGNED) / 100.0), 2)    AS billed,
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0), 2)      AS paid,
    ROUND(SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0), 2) AS
client_collections,
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0), 2) AS total_revenue,
    ROUND(SUM(CAST(ch.adjusted_cents AS SIGNED) / 100.0), 2)  AS adjustments,
    ROUND(SUM(CAST(ch.denied_cents AS SIGNED) / 100.0), 2)    AS denials,
    ROUND(SUM(CAST(ch.balance_cents AS SIGNED) / 100.0), 2)   AS balance,
    ROUND(
        (SUM(CAST(ch.paid_cents AS SIGNED)) + SUM(CAST(ch.copaid_cents AS SIGNED))
         + SUM(CAST(ch.client_paid_cents AS SIGNED)))
        * 100.0 / NULLIF(SUM(CAST(ch.amount_cents AS SIGNED)), 0), 1
    )                                                           AS
collection_rate_pct
FROM procedures AS pr
INNER JOIN fees                AS fe   ON pr.fee_id = fe.id
INNER JOIN superbills          AS sb   ON pr.superbill_id = sb.id
INNER JOIN insurance_policies  AS ip   ON sb.first_insurance_id = ip.id
INNER JOIN companies           AS pa   ON ip.payor_id = pa.id
INNER JOIN people              AS pp   ON sb.attending_provider_id =
pp.identifiable_id
                                       AND pp.identifiable_type = 'User'
INNER JOIN locations           AS loc  ON sb.location_id = loc.id
LEFT  JOIN charges_procedures  AS chpr ON pr.id = chpr.procedure_id
LEFT  JOIN charges             AS ch   ON chpr.charge_id = ch.id
LEFT  JOIN encounter_forms     AS ef   ON sb.encounter_form_id = ef.id
LEFT  JOIN encounters          AS en   ON ef.encounter_id = en.id
LEFT  JOIN appointments        AS app  ON en.appointment_id = app.id
WHERE sb.reported_start BETWEEN CAST(@from_date AS DATETIME)
                            AND CAST(CONCAT(@until_date, ' 23:59:59') AS DATETIME)
GROUP BY loc.name
ORDER BY total_revenue DESC;
```

---

5. Provider Performance

```sql
-- Revenue and productivity by provider (attending clinician)
--
-- Provider name is resolved through:
--   superbills.attending_provider_id → people.identifiable_id (where type = 'User')
--   people.id → names.person_id (get latest name record via subquery)
--
-- The names table stores name history; we always want the most recent
-- name record (highest id) for each person.
SELECT
    CONCAT(nm.first, ' ', nm.last)                              AS provider,
    COUNT(DISTINCT ef.id)                                       AS encounters,
    COUNT(DISTINCT app.id)                                      AS appointments,
    COUNT(DISTINCT sb.client_id)                                AS unique_clients,
    ROUND(SUM(CAST(fe.rate_cents AS SIGNED) / 100.0 * pr.units), 2) AS fee_charges,
    ROUND(SUM(CAST(ch.amount_cents AS SIGNED) / 100.0), 2)    AS billed,
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0), 2)      AS paid,
    ROUND(SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0), 2) AS
client_collections,
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
        + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0), 2) AS total_revenue,
    ROUND(SUM(CAST(ch.adjusted_cents AS SIGNED) / 100.0), 2)  AS adjustments,
    ROUND(SUM(CAST(ch.denied_cents AS SIGNED) / 100.0), 2)    AS denials,
    ROUND(SUM(CAST(ch.balance_cents AS SIGNED) / 100.0), 2)   AS balance,
    -- Revenue per encounter
    ROUND(
        (SUM(CAST(ch.paid_cents AS SIGNED) / 100.0)
         + SUM(CAST(ch.copaid_cents AS SIGNED) / 100.0)
         + SUM(CAST(ch.client_paid_cents AS SIGNED) / 100.0))
        / NULLIF(COUNT(DISTINCT ef.id), 0), 2
    )                                                           AS
revenue_per_encounter
FROM procedures AS pr
INNER JOIN fees                AS fe   ON pr.fee_id = fe.id
INNER JOIN superbills          AS sb   ON pr.superbill_id = sb.id
INNER JOIN insurance_policies  AS ip   ON sb.first_insurance_id = ip.id
INNER JOIN companies           AS pa   ON ip.payor_id = pa.id
-- Provider: people table uses polymorphic link (identifiable_type = 'User')
INNER JOIN people              AS pp   ON sb.attending_provider_id =
pp.identifiable_id
                                       AND pp.identifiable_type = 'User'
-- Get the latest name for this provider (names table stores history)
INNER JOIN names               AS nm   ON nm.id = (
                SELECT names.id FROM names
                 WHERE names.person_id = pp.id
              ORDER BY id DESC LIMIT 1
           )
```

```sql
INNER JOIN locations           AS loc  ON sb.location_id = loc.id
LEFT  JOIN charges_procedures  AS chpr ON pr.id = chpr.procedure_id
LEFT  JOIN charges             AS ch   ON chpr.charge_id = ch.id
LEFT  JOIN encounter_forms     AS ef   ON sb.encounter_form_id = ef.id
LEFT  JOIN encounters          AS en   ON ef.encounter_id = en.id
LEFT  JOIN appointments        AS app  ON en.appointment_id = app.id
WHERE sb.reported_start BETWEEN CAST(@from_date AS DATETIME)
                            AND CAST(CONCAT(@until_date, ' 23:59:59') AS DATETIME)
GROUP BY provider
ORDER BY total_revenue DESC;
```

---

6. Clients per Zip Code

```sql
-- Client distribution by zip code
--
-- Uses the reportables table — a denormalized/materialized view that
-- pre-joins encounter_forms with client demographics (including zip).
-- This avoids the complex multi-join to addresses table.
--
-- SUBSTRING(zip, 1, 5) normalizes zip codes (strips +4 extensions).
SELECT
    IFNULL(SUBSTRING(r.client_zip, 1, 5), 'N/A')              AS zip_code,
    COUNT(DISTINCT r.client_id)                                AS unique_clients,
    COUNT(DISTINCT r.encounter_form_id)                        AS encounters,
    -- Average encounters per client in this zip
    ROUND(COUNT(DISTINCT r.encounter_form_id)
          / COUNT(DISTINCT r.client_id), 1)                    AS
avg_encounters_per_client
FROM reportables AS r
WHERE r.encounter_form_id IS NOT NULL
  AND r.client_id IS NOT NULL
  AND r.date_of_service BETWEEN CAST(@from_date AS DATE)
                            AND CAST(@until_date AS DATE)
GROUP BY zip_code
ORDER BY unique_clients DESC;
```

---

7. Diagnosis per Zip Code

```sql
-- Diagnosis distribution by client zip code
--
-- Joins reportables (for zip + encounter data) with diagnoses/icds (for ICD-10
codes).
-- diagnoses table links clients to ICD codes; each client can have multiple
diagnoses.
-- icds table contains the code and description.
--
-- This cross-references WHERE clients live with WHAT they're diagnosed with,
-- useful for population health mapping.
```

```sql
SELECT
    IFNULL(SUBSTRING(r.client_zip, 1, 5), 'N/A')              AS zip_code,
    icd.code                                                   AS icd_code,
    icd.description                                            AS diagnosis,
    COUNT(DISTINCT r.client_id)                                AS unique_clients,
    COUNT(DISTINCT r.encounter_form_id)                        AS encounters
FROM reportables AS r
-- diagnoses: each row = one diagnosis assigned to a client
INNER JOIN diagnoses AS dxs ON dxs.client_id = r.client_id
-- icds: lookup table for ICD-10 codes and descriptions
INNER JOIN icds      AS icd ON dxs.icd_id = icd.id
WHERE r.encounter_form_id IS NOT NULL
  AND r.client_id IS NOT NULL
  AND dxs.deleted_at IS NULL
  AND r.date_of_service BETWEEN CAST(@from_date AS DATE)
                            AND CAST(@until_date AS DATE)
GROUP BY zip_code, icd.code, icd.description
ORDER BY zip_code, unique_clients DESC;
```

7b. Top 10 Diagnoses per Zip Code (pivoted for heatmap)

```sql
-- Top 10 diagnoses overall, with client counts per zip code
-- Useful for a QuickSight heatmap or pivot table
SELECT
    icd.code                                                   AS icd_code,
    icd.description                                            AS diagnosis,
    IFNULL(SUBSTRING(r.client_zip, 1, 5), 'N/A')              AS zip_code,
    COUNT(DISTINCT r.client_id)                                AS unique_clients
FROM reportables AS r
INNER JOIN diagnoses AS dxs ON dxs.client_id = r.client_id
INNER JOIN icds      AS icd ON dxs.icd_id = icd.id
WHERE r.encounter_form_id IS NOT NULL
  AND r.client_id IS NOT NULL
  AND dxs.deleted_at IS NULL
  AND r.date_of_service BETWEEN CAST(@from_date AS DATE)
                            AND CAST(@until_date AS DATE)
  -- Limit to top 10 diagnoses by overall client count
  AND icd.code IN (
      SELECT icd2.code
      FROM reportables r2
      INNER JOIN diagnoses dxs2 ON dxs2.client_id = r2.client_id
      INNER JOIN icds      icd2 ON dxs2.icd_id = icd2.id
      WHERE r2.encounter_form_id IS NOT NULL
        AND r2.client_id IS NOT NULL
        AND dxs2.deleted_at IS NULL
        AND r2.date_of_service BETWEEN CAST(@from_date AS DATE) AND CAST(@until_date
AS DATE)
      GROUP BY icd2.code
      ORDER BY COUNT(DISTINCT r2.client_id) DESC
      LIMIT 10
  )
```

```
GROUP BY icd.code, icd.description, zip_code
ORDER BY icd_code, unique_clients DESC;
```

---

8. Suggested Analyses (5 Additional Graphs)

8a. Monthly Revenue Trend (Line Chart)

```
-- Monthly revenue trend — billed vs paid vs denied
-- Graph type: Multi-line chart (3 lines over time)
-- Purpose: Track revenue cycle health over time, spot collection delays or denial
spikes
SELECT
    DATE_FORMAT(sb.reported_start, '%Y-%m')                    AS month,
    COUNT(DISTINCT sb.id)                                      AS superbills,
    COUNT(DISTINCT sb.client_id)                               AS unique_clients,
    ROUND(SUM(CAST(ch.amount_cents AS SIGNED) / 100.0), 2)    AS billed,
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0), 2)      AS paid,
    ROUND(SUM(CAST(ch.denied_cents AS SIGNED) / 100.0), 2)    AS denied,
    ROUND(SUM(CAST(ch.adjusted_cents AS SIGNED) / 100.0), 2)  AS adjusted,
    ROUND(SUM(CAST(ch.balance_cents AS SIGNED) / 100.0), 2)   AS balance
FROM procedures AS pr
INNER JOIN fees                AS fe   ON pr.fee_id = fe.id
INNER JOIN superbills          AS sb   ON pr.superbill_id = sb.id
LEFT  JOIN charges_procedures  AS chpr ON pr.id = chpr.procedure_id
LEFT  JOIN charges             AS ch   ON chpr.charge_id = ch.id
WHERE sb.reported_start BETWEEN CAST(@from_date AS DATETIME)
                            AND CAST(CONCAT(@until_date, ' 23:59:59') AS DATETIME)
GROUP BY month
ORDER BY month;
```

8b. Payor Mix — Encounters by Insurance Type (Donut Chart)

```
-- Payor mix distribution — what percentage of encounters each payor represents
-- Graph type: Donut/pie chart
-- Purpose: Understand insurance mix for contract negotiations and revenue
forecasting
SELECT
    pa.name                                                    AS payor,
    COUNT(DISTINCT ef.id)                                      AS encounters,
    COUNT(DISTINCT sb.client_id)                               AS unique_clients,
    -- Percentage of total encounters
    ROUND(COUNT(DISTINCT ef.id) * 100.0 / (
        SELECT COUNT(DISTINCT ef2.id)
        FROM superbills sb2
        INNER JOIN encounter_forms ef2 ON sb2.encounter_form_id = ef2.id
        WHERE sb2.reported_start BETWEEN CAST(@from_date AS DATETIME)
                                     AND CAST(CONCAT(@until_date, ' 23:59:59') AS
DATETIME)
    ), 1)                                                      AS pct_of_encounters
```

```
FROM superbills AS sb
INNER JOIN insurance_policies  AS ip   ON sb.first_insurance_id = ip.id
INNER JOIN companies           AS pa   ON ip.payor_id = pa.id
LEFT  JOIN encounter_forms     AS ef   ON sb.encounter_form_id = ef.id
WHERE sb.reported_start BETWEEN CAST(@from_date AS DATETIME)
                            AND CAST(CONCAT(@until_date, ' 23:59:59') AS DATETIME)
GROUP BY pa.name
ORDER BY encounters DESC;
```

8c. Denial Rate by Payor (Horizontal Bar Chart)

```
-- Denial rate comparison across payors
-- Graph type: Horizontal stacked bar chart (paid vs denied vs adjusted per payor)
-- Purpose: Identify payors with high denial rates for follow-up and appeals process
improvement
SELECT
    pa.name                                                      AS payor,
    ROUND(SUM(CAST(ch.amount_cents AS SIGNED) / 100.0), 2)    AS billed,
    ROUND(SUM(CAST(ch.paid_cents AS SIGNED) / 100.0), 2)      AS paid,
    ROUND(SUM(CAST(ch.denied_cents AS SIGNED) / 100.0), 2)    AS denied,
    ROUND(SUM(CAST(ch.adjusted_cents AS SIGNED) / 100.0), 2)  AS adjusted,
    -- Denial rate = denied / billed
    ROUND(SUM(CAST(ch.denied_cents AS SIGNED))
          * 100.0 / NULLIF(SUM(CAST(ch.amount_cents AS SIGNED)), 0), 1) AS
denial_rate_pct,
    -- Collection rate = (paid + copaid + client) / billed
    ROUND(
        (SUM(CAST(ch.paid_cents AS SIGNED)) + SUM(CAST(ch.copaid_cents AS SIGNED))
         + SUM(CAST(ch.client_paid_cents AS SIGNED)))
        * 100.0 / NULLIF(SUM(CAST(ch.amount_cents AS SIGNED)), 0), 1
    )                                                            AS
collection_rate_pct
FROM procedures AS pr
INNER JOIN fees                AS fe   ON pr.fee_id = fe.id
INNER JOIN superbills          AS sb   ON pr.superbill_id = sb.id
INNER JOIN insurance_policies  AS ip   ON sb.first_insurance_id = ip.id
INNER JOIN companies           AS pa   ON ip.payor_id = pa.id
LEFT  JOIN charges_procedures  AS chpr ON pr.id = chpr.procedure_id
LEFT  JOIN charges             AS ch   ON chpr.charge_id = ch.id
WHERE sb.reported_start BETWEEN CAST(@from_date AS DATETIME)
                            AND CAST(CONCAT(@until_date, ' 23:59:59') AS DATETIME)
GROUP BY pa.name
HAVING billed > 0
ORDER BY denial_rate_pct DESC;
```

8d. Average Days to Payment by Payor (Bar Chart)

```
-- Average time from date of service to first payment, by payor
-- Graph type: Vertical bar chart
```

```sql
-- Purpose: Identify slow-paying payors; benchmark against timely filing limits
--
-- billcases.date_of_service = when the service happened
-- billcases.first_pay_date  = when first payment was received
-- The difference = days to collect
SELECT
    pa.name                                                    AS payor,
    COUNT(DISTINCT bc.id)                                      AS
billcases_with_payment,
    ROUND(AVG(DATEDIFF(bc.first_pay_date, DATE(bc.date_of_service))), 0) AS
avg_days_to_payment,
    MIN(DATEDIFF(bc.first_pay_date, DATE(bc.date_of_service))) AS min_days,
    MAX(DATEDIFF(bc.first_pay_date, DATE(bc.date_of_service))) AS max_days
FROM billcases AS bc
-- first_payor_id links directly to the primary payor on the billcase
INNER JOIN companies AS pa ON bc.first_payor_id = pa.id
WHERE bc.deleted_at IS NULL
  AND bc.first_pay_date IS NOT NULL
  AND bc.date_of_service BETWEEN CAST(@from_date AS DATETIME)
                              AND CAST(CONCAT(@until_date, ' 23:59:59') AS
DATETIME)
GROUP BY pa.name
ORDER BY avg_days_to_payment DESC;
```

8e. Program Enrollment Census Over Time (Stacked Area Chart)

```sql
-- Monthly active enrollment count by program
-- Graph type: Stacked area chart
-- Purpose: Track program growth/decline, capacity planning
--
-- A client is "active" in a program for a given month if:
--   - approved_at is before end of month
--   - closure_reason is NULL or closed after the month
--   - denied_at is NULL
SELECT
    months.month,
    p.name                                                     AS program,
    COUNT(DISTINCT pe.client_id)                               AS active_clients
FROM (
    -- Generate month series within the date range
    SELECT DISTINCT DATE_FORMAT(date_of_service, '%Y-%m-01') AS month
    FROM reportables
    WHERE date_of_service BETWEEN CAST(@from_date AS DATE) AND CAST(@until_date AS
DATE)
) AS months
CROSS JOIN programs AS p
-- Join enrollments that were active during each month
INNER JOIN program_enrollments AS pe ON pe.program_id = p.id
    -- Approved before end of this month
    AND pe.approved_at IS NOT NULL
    AND pe.approved_at <= LAST_DAY(months.month)
```

```sql
    -- Not closed before this month (or never closed)
    AND (pe.closure_reason IS NULL
         OR pe.closed_by IS NULL
         OR pe.updated_at >= months.month)
    -- Not denied
    AND pe.denied_at IS NULL
WHERE p.active = 1
GROUP BY months.month, p.name
HAVING active_clients > 0
ORDER BY months.month, active_clients DESC;
```

Notes for Data Analyst

Key Relationships

```
Encounter Flow:
  appointment → encounter → encounter_form → superbill → procedures → fees
                                           → billcase  → charges →
payments/denials/adjustments

Superbill is the pivot:
  superbill.first_insurance_id   → insurance_policies → companies (payor)
  superbill.attending_provider_id → people (identifiable_type='User') → names
  superbill.location_id          → locations
  superbill.client_id            → people (type='Client')
  superbill.reported_start       → date of service (use for time filtering)
```

Financial Column Convention

All money stored as `*_cents` (integers). Always `CAST(x_cents AS SIGNED) / 100.0` for dollars.

Revenue Formula

`revenue = paid + copaid + client_paid` (matches Bear's metric pattern)

Soft Deletes

Most tables use `deleted_at IS NULL` for active records. Always include this filter.

Name Resolution

Names are in a separate `names` table (supports history). Always get the latest via `ORDER BY id DESC LIMIT 1` subquery on `names.person_id`.